RegNo: 223026689 **date:29th/03/2026**
UR CST
COMPUTER ENGINEERING
YEAR 3
SEMESTER II
GAKO CAMPUS

Lab 5: Posts Manager with SQLite:

Offline Posts Manager — SQLite Local Storage

Local Database: SQLite via sqflite package

LAB 5: Local storage in Flutter

Question 1: Which dependencies ~~dat~~ used and why? Explain why SQLite is necessary to store data locally.

Dependency	purpose
SQflite	SQlite plugin for flutter that provides local database storage. It allows crud operations on SQLite databases. It allows storing, retrieving, updating, and deleting data locally
Path_provider	it helps to find the correct directory on the device to store the database.
path	Used to build proper file paths for the database. or you can say, helper for constructing database file paths in a platform-independent way.
Intl	Provide internationalization and date formatting capabilities

Why SQLite is neccessary:

1. Offline Functionality: SQLite store data directly on the device allowing users to access and manage posts without internet connectivity

2. Data persistance: Unlike in memory storage, SQLite ensures data survives app restarts, device reboots and remains available long term

3. Structured data storage: SQLite provide relational database capabilities with tables columns and relationships, making it deal for structured data like posts.

4. Efficient Querying: SQLite support complex queries (search, filtering, sorting) that would be inefficient with simple file storage.

5. Light Weight: SQLite is embedded, requires no server setup and has a small footprint.

Question 2 Handling Database Exceptions

(a) Database Not initialized

- Ensure database is opened before use:

```
if (_database == null) {
    await initDatabase();
}
```

(b) Insert / Update / Delete Errors

- Use try-catch blocks

```
try {
  await db.insert('posts', post.toMap());
} catch (e) {
    print("Insert error: $e");
}
```

c. Invalid or Corrupted Data

- Validate inputs before Saving :

```
if (title.isEmpty || content.isEmpty) {
  throw Exception("Invalid data");
}
```

- Handle unexpected data when reading.

```
try {
  final result = await db.query('posts');
} catch (e) {
  print("Read error : $e");
}
```

Question 3 Explain SQLite in Flutter

a. Database vs Table

Concept	Explanation
Database	A container that holds multiple tables. In our app, posts_database.db is the database file.
Table	A structured collection of data within a database. Our posts table stores individual posts with columns : id, title, content, created_at, updated_at

b. CRUD operations

Create (insert):

```dart
await db.insert('post', post.toMap());
```

Read (fetch):

```dart
await db.query('posts');
```

Update:

```dart
await db.update('post', post.toMap(), where: 'id = ?',
    whereArgs: [id]);
```

Delete:

```dart
await db.delete('posts', where: 'id = ?', whereArgs: [id]);
```

c. Asynchronous Interaction in Flutter

- SQlite operations take time → handled using async/await.

Example:

```dart
Future<List<post>> getPosts() async {
    final db = await database;
    final maps = await db.query('posts');
    return maps.map(e) => Post.fromMap(e).toList();
}.
```

This prevents the UI from freezing and keeps the app responsive.

PART 2 — Application Screenshots

The following screenshots demonstrate all five CRUD features of the Offline Posts Manager application running on an Android emulator. All data is stored and retrieved from a local SQLite database — no internet connection is required.

Required Screenshots — Checklist

#	Screen Name	Feature Demonstrated
1	**Home Screen**	View all posts stored in SQLite database (READ ALL)
2	**Post Detail Screen**	Read the full content of one post (READ ONE)
3	**Add Post Screen**	Empty form to create a new post (CREATE)
4	**Edit Post Screen**	Pre-filled form to modify an existing post (UPDATE)
5	**Delete Confirmation**	Dialog to confirm deletion of a post (DELETE)
6	**Empty State**	Home screen when the database has no posts yet

Screenshot 1 — Home Screen: View All Posts

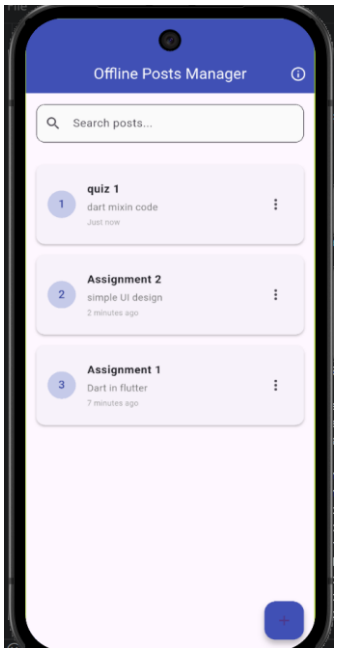

Figure 1: Offline Posts Manager — Home screen displaying all posts loaded from SQLite

Screenshot 2 — Post Detail Screen

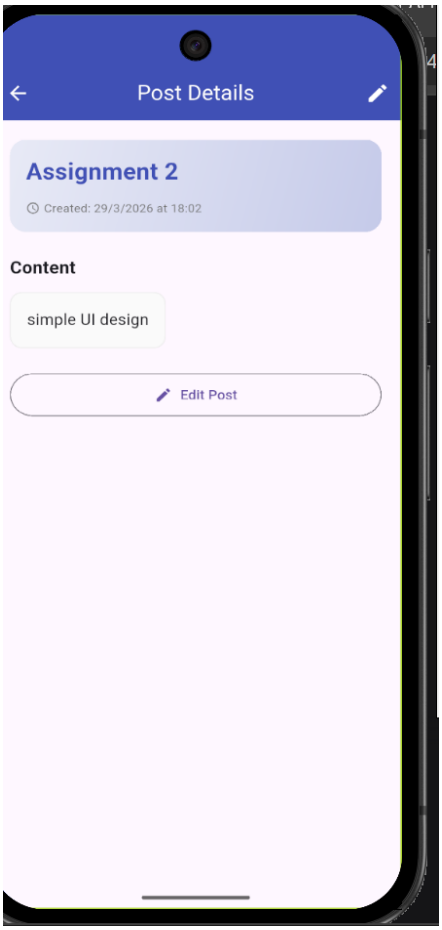

Figure 2: Post Detail Screen showing the complete title and body of a selected post

What this screen shows: The full details of a single post selected from the home screen list. It displays the post ID, complete title, and full body text without truncation.

Database operation: The post data is passed directly from the home screen as a Post object — no additional database query is needed. This is efficient because the data was already loaded by the home screen's SELECT * FROM posts query.

Navigation: The AppBar has a back arrow (←) to return to the home screen. An edit icon (pencil) in the top-right corner navigates to the Edit Post screen for this specific post.

Post ID display: The post's auto-incremented SQLite id field is displayed, confirming that SQLite successfully assigned a unique identifier when the post was inserted into the database.

Offline confirmation: All data on this screen comes from the local SQLite database, not from any network request. This confirms the offline-first design requirement of the lab.

Screenshot 3 — Add New Post Screen

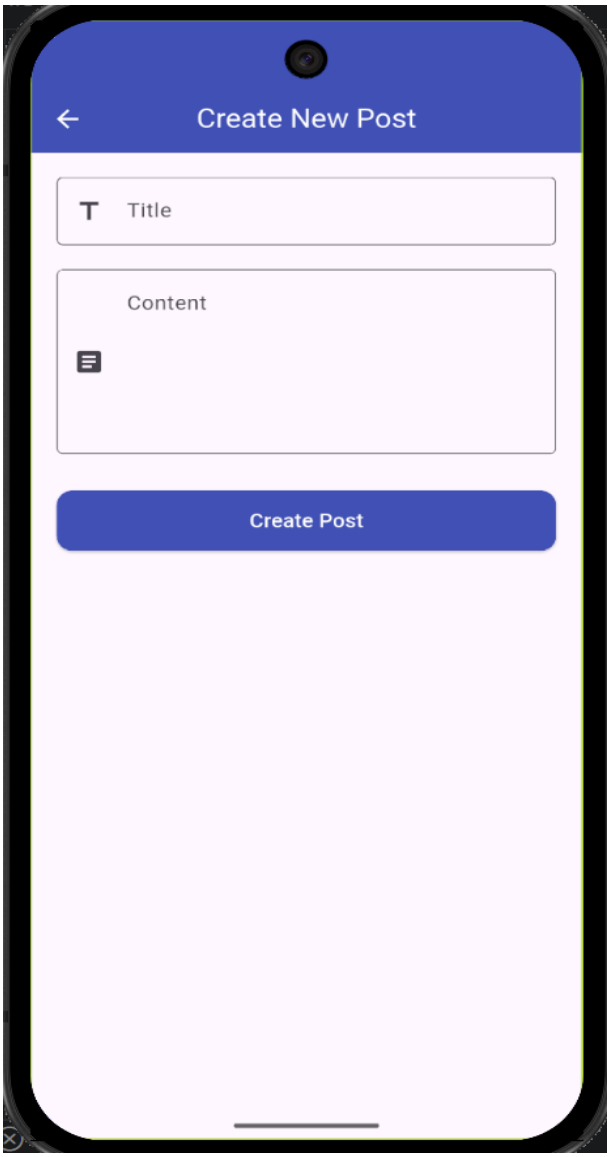

Figure 3: Add Post Screen with empty form fields ready for user input

What this screen shows: The Create Post screen with an empty form. The user fills in the Title and Body fields and taps Save Post to insert a new record into the SQLite database.

Database operation (INSERT): When Save Post is tapped, the app calls DatabaseHelper.insertPost(post), which executes INSERT INTO posts (title, body) VALUES (?, ?). SQLite automatically assigns a unique auto-incremented id to the new post.

Form validation: Before inserting into the database, the form validates that neither the Title nor Body field is empty. If validation fails, an error message appears and no database operation is performed.

sqflite INSERT method: The sqflite package's db.insert('posts', post.toMap()) method is used. It returns the new row's id as an integer, confirming the successful insertion.

After saving: On success, the app navigates back to the Home Screen and calls _loadPosts() to re-query the database, so the new post immediately appears in the list.

Screenshot 4 — Edit Post Screen

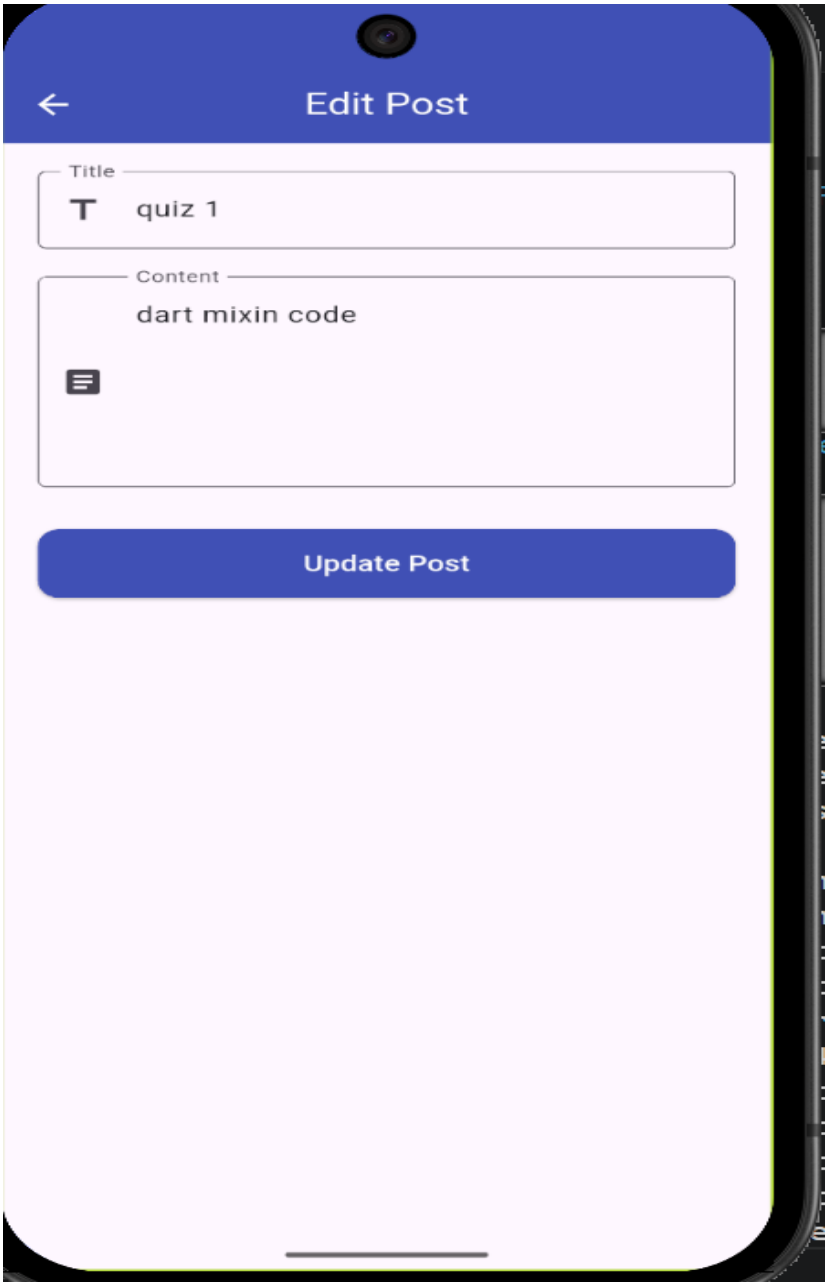

Figure 4: Edit Post Screen with existing post data pre-filled in the form fields

What this screen shows: The Edit Post screen with the selected post's current data pre-loaded into the form fields. The user can modify the Title and/or Body and tap Update Post to save the changes.

Database operation (UPDATE): When Update Post is tapped, the app calls DatabaseHelper.updatePost(post), which executes UPDATE posts SET title = ?, body = ? WHERE id = ?. The post's existing id is used to identify exactly which row to modify.

Pre-filling the form: The TextEditingControllers are initialized with the existing post data (post.title and post.body) in the initState() method. This ensures the user sees the current values and can edit only what they want to change.

sqflite UPDATE method: The sqflite package's db.update('posts', post.toMap(), where: 'id = ?', whereArgs: [post.id]) method is used. It returns the number of rows affected, which should be 1 on success.

After updating: On success, the app navigates back and the home screen refreshes its list via _loadPosts(), showing the updated post title and body immediately.

Screenshot 5 — Delete Confirmation Dialog

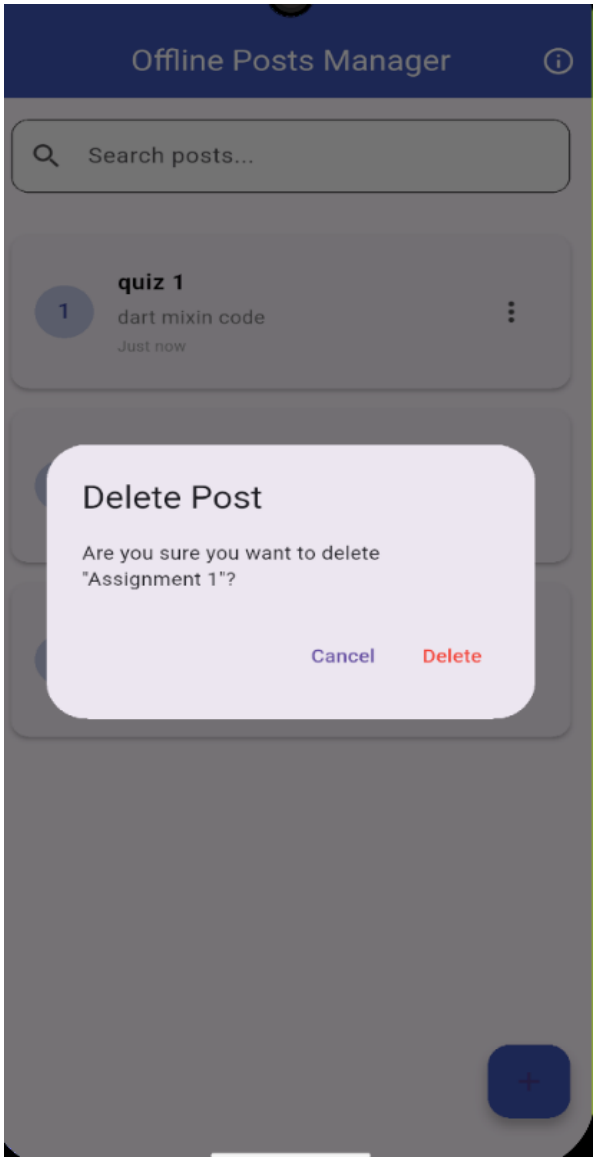

Figure 5: Delete Confirmation Dialog appearing as a modal over the Posts list

What this screen shows: A confirmation AlertDialog that appears when the user taps the delete icon on a post. It asks the user to confirm before permanently removing the post from the SQLite database.

Why a confirmation dialog: Deleting from a local SQLite database is irreversible — there is no remote server to recover data from. The confirmation step prevents accidental data loss, which is especially important for offline-only storage.

Database operation (DELETE): If the user taps 'Delete', the app calls DatabaseHelper.deletePost(id), which executes DELETE FROM posts WHERE id = ?. The specific post's id is passed as a parameter to target only that row.

sqflite DELETE method: The sqflite package's db.delete('posts', where: 'id = ?', whereArgs: [id]) method is used. It returns the number of rows deleted, which should be 1 on success.

After deleting: The dialog closes, a SnackBar shows 'Post deleted', and the home screen list refreshes via _loadPosts(), removing the deleted post from the UI immediately.

Cancel action: If the user taps 'Cancel', Navigator.pop() closes the dialog and no database operation is performed — the post remains in the database unchanged.

Screenshot 6 — Empty State (No Posts in Database)

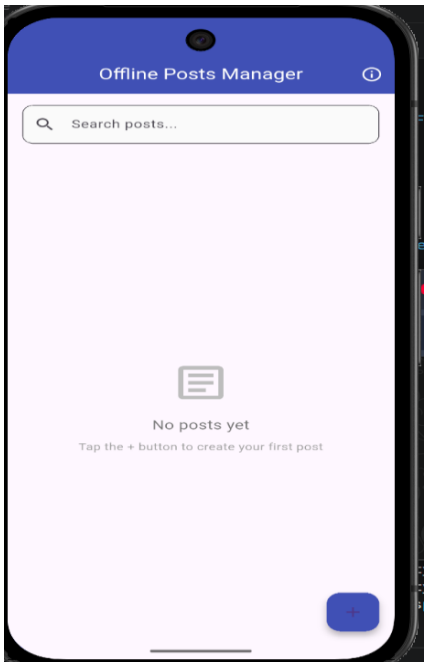

Figure 6: Empty State screen shown when no posts are stored in the SQLite database

What this screen shows: The home screen when the SQLite database contains zero posts. Instead of showing an empty ListView, the app shows a helpful message guiding the user to add their first post.

Database state: The SELECT * FROM posts query returned an empty list ([]). This happens on first launch (before any posts are inserted) or after all posts have been deleted.

Empty state UI: The app checks if the loaded list is empty using an if (posts.isEmpty) condition and renders a Center widget with an icon and message like 'No posts yet. Tap + to add one.' This is a UX best practice.

SQLite initialization: Even with an empty posts table, the database itself is already open and the table exists. The DatabaseHelper class creates the table with CREATE TABLE IF NOT EXISTS on first launch, so it is always ready to accept new inserts.

Importance for the lab: This screenshot proves that the SQLite database and table were successfully created and initialized, and that the app handles the empty state gracefully without crashes or errors.

Summary of CRUD Operations Demonstrated

Fig.	CRUD	SQLite Operation	Screen
Fig. 1	**READ ALL**	SELECT * FROM posts	Home Screen — full post list
Fig. 2	**READ ONE**	Data passed from list query	Post Detail Screen
Fig. 3	**CREATE**	INSERT INTO posts (...)	Add Post — empty form
Fig. 4	**UPDATE**	UPDATE posts SET ... WHERE id=?	Edit Post — pre-filled form
Fig. 5	**DELETE**	DELETE FROM posts WHERE id=?	Delete Confirmation Dialog
Fig. 6	**READ ALL**	SELECT * FROM posts → empty []	Empty State — no posts